

FINANCIAL MANAGEMENT SYSTEM – SQL PROJECT DOCUMENTATION

- **Prepared By**

Shaista Anjum

- **Course**

Data science with Gen AI

- **Project Duration**

6 Days

- **TECHNOLOGY USED**

Oracle SQL

- **Institute**

Fortune IT

- **Submitted To:**

NAYEEM SIR

Financial Management System using Oracle SQL

This project is developed to understand the real-time usage of Oracle SQL in financial and banking systems. In this project, different tables such as Customers, Accounts, Transactions, Loans, and Employees were created. Various operations such as data insertion, data cleaning, joins, aggregate functions, TCL commands, DCL commands, ALTER operations, and report generation were performed.

This project helped in improving practical SQL knowledge and understanding how financial data is managed in databases. The project also provided knowledge about relationships between tables, transaction management, and database security.

• Introduction

The Financial Management System project is developed using Oracle SQL. This project is used to manage customer details, account details, **transaction records, loan information, and employee data**. The project includes important SQL concepts such as DDL commands, DML commands, TCL commands, DCL commands, data cleaning, joins, aggregate functions, and report generation.

This project helped in understanding database creation, data management, query writing, and handling real-time database operations.

The project covers important SQL concepts such as:

- DDL Commands
- DML Commands
- TCL Commands
- DCL Commands
- Data Cleaning
- Data Modeling
- Joins
- Aggregate Functions
- Constraints
- Report Generation

This project improved practical SQL knowledge and understanding of how financial data is stored, managed, updated, and secured in real-world database systems.

Objectives

- To understand database management concepts
- To learn Oracle SQL commands
- To create and manage tables
- To perform data cleaning operations
- To generate reports using joins and aggregate functions
- To understand transaction management using TCL commands
- To understand permission management using DCL commands
- To improve practical SQL skills

Database Tables Used

Table Name	Purpose
Customers	Stores customer information
Accounts	Stores account details
Transactions	Stores transaction history
Loans	Stores loan details

Module 1 – Database Creation

- **Definition**

Table creation is done using DDL (Data Definition Language) commands.

Tables are used to store data in rows and columns.

- **Customers Table**

```
CREATE TABLE Customers (customer_id INT PRIMARY KEY, customer_name  
VARCHAR2(100) NOT NULL, email VARCHAR2(100), phone VARCHAR2(15), city  
VARCHAR2(50));
```

Explanation

- PRIMARY KEY uniquely identifies records
- NOT NULL ensures mandatory values
- VARCHAR2 stores text values in Oracle SQL

Accounts Table

```
CREATE TABLE Accounts (account_id INT PRIMARY KEY, customer_id  
INT, account_type VARCHAR2(50), balance NUMBER(10,2), FOREIGN  
KEY(customer_id) REFERENCES Customers(customer_id));
```

Explanation

- FOREIGN KEY creates relationship between tables
- NUMBER datatype stores numeric values

Transactions Table

```
CREATE TABLE Transactions(transaction_id INT PRIMARY KEY, account_id INT,  
transaction_type VARCHAR2(50), amount NUMBER(10,2), transaction_date  
DATE, FOREIGN KEY(account_id) REFERENCES Accounts(account_id));
```

Loans Table

```
CREATE TABLE Loans (loan_id INT PRIMARY KEY, customer_id INT, loan_amount
NUMBER(10,2), loan_status VARCHAR2(50), FOREIGN
KEY(customer_id) REFERENCES Customers(customer_id));
```

Employees Table

```
CREATE TABLE Employees(employee_id INT PRIMARY KEY, employee_name
VARCHAR2(100), department VARCHAR2(50), salary NUMBER(10,2));
```

EXECUTION

```
CUSTOMER_NAME          NOT NULL VARCHAR2(100)
EMAIL                  VARCHAR2(100)
PHONE                  VARCHAR2(15)
CITY                   VARCHAR2(50)

SQL> desc accounts;
Name                   Null?    Type
-----
ACCOUNT_ID             NOT NULL NUMBER
CUSTOMER_ID            NUMBER(38)
ACCOUNT_TYPE            VARCHAR2(50)
BALANCE                NUMBER(10,2)

SQL> desc transactions;
ERROR:
ORA-04043: object transactions does not exist

SQL> desc transactions;
Name                   Null?    Type
-----
TRANSACTION_ID         NOT NULL NUMBER
ACCOUNT_ID             NUMBER
TRANSACTION_TYPE        VARCHAR2(50)
AMOUNT                 NUMBER(10,2)
TRANSACTION_DATE        DATE

SQL> desc loans;
Name                   Null?    Type
-----
LOAN_ID                NOT NULL NUMBER
CUSTOMER_ID            NUMBER
LOAN_AMOUNT            NUMBER(10,2)
LOAN_STATUS            VARCHAR2(50)
```

```
SQL> desc employees;
Name                   Null?    Type
-----
EMPLOYEE_ID            NOT NULL NUMBER
EMPLOYEE_NAME          VARCHAR2(100)
DEPARTMENT              VARCHAR2(50)
SALARY                 NUMBER(10,2)

SQL>
```

Module 2 – Data Modeling

Definition

Data modeling is the process of creating relationships between tables using primary keys and foreign keys.

Relationship Example

- One customer can have many accounts
- One account can have many transactions
- One customer can have many loans

Explanation

In this module, relationships were created between Customers, Accounts, Transactions, and Loans tables using primary keys and foreign keys.

1. One customer can have multiple accounts. How will you design the relationship?

- **Use One-to-Many Relationship.**
- **One customer → Many accounts**
- **Customer_ID is Primary Key in Customers table**
- **Customer_ID is Foreign Key in Accounts table**

2. How will you connect transactions with accounts?

- **Using Account_ID.**
- **Account_ID is Primary Key in Accounts**
- **Account_ID is Foreign Key in Transactions**

3. Which columns should be Primary Keys?

- **Customers → Customer_ID**
- **Accounts → Account_ID**
- **Transactions → Transaction_ID**
- **Loans → Loan_ID**
- **Employees → Employee_ID**

4. Which columns should not allow NULL values?

- **Customer_Name , Employee Name, Account_Type , Primary key colum**

Module 3 – Raw Data Insertion

Definition

Data insertion is done using INSERT command. It is used to add records into tables.

- Insert Data into Customers Table

```
INSERT INTO Customers VALUES(1, 'Ayesha Khan', 'ayesha@gmail.com',  
'9876543210', 'Hyderabad');INSERT INTO Customers VALUES(2, 'Rahul Sharma',  
NULL, '9876501234', 'Mumbai');
```

Explanation

- NULL values were intentionally inserted for data cleaning practice.

EXECUTION

```
Connected.  
SQL> INSERT INTO Customers VALUES  
2 (1, 'Ayesha Khan', 'ayesha@gmail.com', '9876543210', 'Hyderabad');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Customers VALUES  
2 (2, 'Rahul Sharma', NULL, '9876501234', 'Mumbai');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Customers VALUES  
2 (3, 'Noor Fatima', 'noor@gmail.com', NULL, 'Delhi');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Customers VALUES  
2 (4, 'Ali Ahmed', 'ali@gmail.com', '9999999999', 'hyderabad');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Customers VALUES  
2 (5, 'Sneha Reddy', 'sneha@gmail.com', '8888888888', 'Chennai');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Customers VALUES  
2 (6, 'Ayesha Khan', 'ayesha@gmail.com', '9876543210', 'Hyderabad');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Customers VALUES  
2 (7, 'Rohan Verma', NULL, '7777777777', 'Pune');  
  
1 row created.  
  
SQL>
```

```

1 row created.

SQL>
SQL> INSERT INTO Customers VALUES
  2 (8, 'Meera Joshi', 'meera@gmail.com', NULL, 'Bangalore');
1 row created.

SQL>
SQL> INSERT INTO Customers VALUES
  2 (9, 'Arjun Singh', 'arjun@gmail.com', '6666666666', 'Delhi');
1 row created.

SQL>
SQL> INSERT INTO Customers VALUES
  2 (10, 'Sara Ali', 'sara@gmail.com', '5555555555', 'Kolkata');

```

• Insert Data into Accounts Table

```
INSERT INTO Accounts VALUES(101, 1, 'Savings', 50000);
```

```
INSERT INTO Accounts VALUES(103, 3, 'Savings', -2000);
```

Explanation

- Negative balances were inserted for cleaning operations.

EXECUTION

```

SQL> INSERT INTO Accounts VALUES
  2 (101, 1, 'Savings', 50000);
1 row created.

SQL>
SQL> INSERT INTO Accounts VALUES
  2 (102, 2, 'Current', 75000);
1 row created.

SQL>
SQL> INSERT INTO Accounts VALUES
  2 (103, 3, 'Savings', -2000);
1 row created.

SQL>
SQL> INSERT INTO Accounts VALUES
  2 (104, 4, 'Current', 90000);
1 row created.

SQL>
SQL> INSERT INTO Accounts VALUES
  2 (105, 5, 'Savings', 30000);
1 row created.

SQL>
SQL> INSERT INTO Accounts VALUES
  2 (106, 6, 'Savings', 45000);
1 row created.

SQL>
SQL> INSERT INTO Accounts VALUES
  2 (107, 7, 'Current', NULL);
1 row created.

SQL>

```

```
1 row created.

SQL>
SQL> INSERT INTO Accounts VALUES
  2 (108, 8, 'Savings', 120000);

1 row created.

SQL>
SQL> INSERT INTO Accounts VALUES
  2 (109, 9, 'Current', 10000);

1 row created.

SQL>
SQL> INSERT INTO Accounts VALUES
  2 (110, 10, 'Savings', -500);
```

Insert Data into Transactions Table

- **INSERT INTO** Transactions **VALUES**(1001, 101, 'Deposit', 10000, **DATE** '2026-05-01');

INSERT INTO Transactions **VALUES**(1019, 109, 'Deposit', -5000, **DATE** '2026-05-19');

Explanation

- DATE datatype stores transaction dates.
- Invalid transaction amounts were inserted intentionally.

```

SQL> INSERT INTO Transactions VALUES
  2 (1001, 101, 'Deposit', 10000, DATE '2026-05-01');
1 row created.

SQL>
SQL> INSERT INTO Transactions VALUES
  2 (1002, 102, 'Withdraw', 5000, DATE '2026-05-02');
1 row created.

SQL>
SQL> INSERT INTO Transactions VALUES
  2 (1003, 103, 'Deposit', -3000, DATE '2026-05-03');
1 row created.

SQL>
SQL> INSERT INTO Transactions VALUES
  2 (1004, 104, 'Withdraw', 7000, DATE '2026-05-04');
1 row created.

SQL>
SQL> INSERT INTO Transactions VALUES
  2 (1005, 105, 'Deposit', 15000, DATE '2026-05-05');
1 row created.

SQL>
SQL> INSERT INTO Transactions VALUES
  2 (1006, 106, 'Withdraw', 2000, DATE '2026-05-06');
1 row created.

SQL>
SQL> INSERT INTO Transactions VALUES
  2 (1007, 107, 'Deposit', NULL, DATE '2026-05-07');
1 row created.

SQL>

```

```

SQL>
SQL> INSERT INTO Transactions VALUES
  2 (1008, 108, 'Withdraw', 9000, DATE '2026-05-08');
1 row created.

SQL>
SQL> INSERT INTO Transactions VALUES
  2 (1009, 109, 'Deposit', 25000, DATE '2026-05-09');
1 row created.

SQL>
SQL> INSERT INTO Transactions VALUES
  2 (1010, 110, 'Withdraw', -1000, DATE '2026-05-10');
1 row created.

SQL>
SQL> commit;
SP2-0042: unknown command "commit" - rest of line ignored.
SQL> commit;

Commit complete.

SQL>

```

INSERTION DATA INTO LOANS TABLE

INSERT INTO Loans **VALUES**(201, 1, 500000, 'Approved');

INSERT INTO Loans **VALUES**(202, 2, 300000, 'Pending');

EXECUTION

```
> INSERT INTO Loans VALUES
2 (201, 1, 500000, 'Approved');
row created.

>
> INSERT INTO Loans VALUES
2 (202, 2, 300000, 'Pending');
row created.

>
> INSERT INTO Loans VALUES
2 (203, 3, 450000, 'Rejected');
row created.

>
> INSERT INTO Loans VALUES
2 (204, 4, 250000, 'Approved');
row created.

>
> INSERT INTO Loans VALUES
2 (205, 5, NULL, 'Pending');
row created.

> commit;
commit complete.

> _
```

INSERTION DATA INTO EMPLOYEES TABLE

INSERT INTO Employees **VALUES**(1, 'Kiran Kumar', 'Finance', 50000);

INSERT INTO Employees **VALUES**(2, 'Priya Sharma', 'HR', 45000);

EXECUTION

```
> INSERT INTO Employees VALUES
(1, 'Kiran Kumar', 'Finance', 50000);
row created.

>
> INSERT INTO Employees VALUES
(2, 'Priya Sharma', 'HR', 45000);
row created.

>
> INSERT INTO Employees VALUES
(3, 'Ahmed Ali', 'Finance', 55000);
row created.

>
> INSERT INTO Employees VALUES
(4, 'Neha Singh', 'IT', 60000);
row created.

>
> INSERT INTO Employees VALUES
(5, 'Ravi Teja', 'Support', NULL);
row created.

> commit;
commit complete.

>
```

Module 4 – Data Cleaning

Definition

Data cleaning is the process of correcting invalid, duplicate, or missing data.

- **Find NULL Values**

```
SELECT * FROM Customers WHERE email IS NULL;
```

- **Replace NULL Values**

```
UPDATE Customers SET phone = '9999999999' WHERE phone IS NULL;
```

Explanation

This query replaces missing phone numbers.

Remove Negative Balances

```
DELETE FROM Accounts WHERE balance < 0;
```

Explanation

This query removes invalid account balances.

- **Standardize City Names**

```
UPDATE Customers SET city = UPPER(city);
```

Upper function converts city name into uppercase

Execution

```
SQL> SELECT * FROM Customers WHERE Email IS NULL;
CUSTOMER_ID CUSTOMER_NAME CITY          PHONE
-----
EMAIL
-----
102 Ruhi      Mumbai      9123456780

SQL> UPDATE Customers SET Phone = 9999999999 WHERE Phone IS NULL;
2 rows updated.

SQL> DELETE FROM Accounts WHERE Balance < 0;
0 rows deleted.

SQL> SELECT Customer_Name, COUNT(*) FROM Customers GROUP BY Customer_Name HAVING COUNT(*) > 1;
no rows selected

SQL> UPDATE Customers SET City = UPPER(City);
10 rows updated.

SQL> -
```

Module 5 – Data Manipulation

Definition

DML (Data Manipulation Language) commands are used to insert, update, delete, and retrieve data.

- **UPDATE Example**

UPDATE Accounts **SET** balance = balance + 10000**WHERE** account_id = 101;

Explanation

This query updates account balance.

- **INSERT Example**

INSERT INTO Customers **VALUES** (11, 'Imran Khan', 'imran@gmail.com', '9871234567', 'Hyderabad');

- **DELETE Example**

DELETE FROM Accounts **WHERE** balance = 0;

- **MODIFY CUSTOMER CONTACT DETAILS**

UPDATE Customers **SET** phone = '8888888888' **WHERE** customer_id = 3;

- **INCREASE LOAN AMOUNT FOR SELECTED CUSTOMER**

UPDATE Loans **SET** loan_amount = loan_amount + 50000 **WHERE** customer_id IN (1, 2, 4);

```
SQL> UPDATE Accounts
  2 SET Balance = Balance - 5000
  3 WHERE Account_ID = 202;

1 row updated.

SQL> select * from Accounts;

ACCOUNT_ID CUSTOMER_ID ACCOUNT_TYPE BALANCE
-----
201         101 Savings      60000
202         102 Current      70000
203         103 Current      25000
204         104 Savings     110000
205         105 Current      15000
206         106 Savings      82000
207         107 Current      43000
208         108 Savings      67000
209         109 Savings     120000
210         110 Current      35000

10 rows selected.

SQL> INSERT INTO Customers VALUES (111, 'Farah', 'Hyderabad', 9876501234, 'farah@gmail.com');

1 row created.

SQL> UPDATE Customers SET Phone = 9123456789, Email = 'ruhi@gmail.com' WHERE Customer_ID = 102;

1 row updated.

SQL> UPDATE Loans SET Loan_Amount = Loan_Amount + 5000 WHERE Customer_ID = 101;

1 row updated.

SQL> DELETE FROM Accounts WHERE Balance = 0;

0 rows deleted.
```

Module 6 – SQL Queries & Reports

Definition

SQL queries are used to retrieve and analyze data from tables.

- **SELECT Query**

1. **SELECT** * **FROM** Customers;

2. **select** * **from** accounts;

WHERE Clause Example

SELECT * **FROM** Accounts **WHERE** balance > 50000;

SELECT * **FROM** Customers **WHERE** City='HYDERABAD'

- **DISTINCT Keyword Example**

SELECT DISTINCT city **FROM** Customers;

Explanation

DISTINCT removes duplicate values from output.

. JOINS Example

1. **SELECT** C.customer_name, A.account_type, A.balance **FROM** Customers C **INNER JOIN** Accounts A **ON** C.customer_id = A.customer_id;

Explanation

INNER JOIN combines related data from multiple tables.

2. **SELECT** C.customer_name, T.transaction_type, T.amount, T.transaction_date **FROM** Customers C **INNER JOIN** Transactions T **ON** A.account_id = T.account_id;

3. **SELECT** C.customer_name, T.amount **FROM** Customers C **INNER JOIN** Accounts A **ON** C.customer_id = A.customer_id **INNER JOIN** Transactions T **ON** A.account_id = T.account_id **WHERE** T.amount > 10000;

- **Aggregate Function Example**

1. **SELECT SUM**(balance) **AS** total_balance **FROM** Accounts;

Explanation

SUM function calculates total balance.

2.SELECT AVG(balance) AS average_balance **FROM** Accounts;

AVG function calculate average_balance

3.SELECT MAX(balance) AS total_balance **FROM** Accounts;

MAX function calculates total balance

4.SELECT COUNT(*) AS total_CUSTOMER **FROM** CUSTOMERS;

COUNT function calculates TOTAL CUSTOMERS FROM CUSTOMER

EXECUTION

- Display All Customers
- Show All Account Details
- Find Customers from Hyderabad
- Display Accounts with Balance Greater than ₹50,000
- Show Customer Name with Account Details
- Display Transaction History for Each Customer
- Find Customers Who Made Transactions Above ₹10,000
- Show Account Type-wise Total Balance
- Find Total Bank Balance
- Find Average Account Balance
- Find Maximum Transaction Amount
- Count Total Customers

```
Run SQL Command Line

SQL> SELECT * FROM Customers;

CUSTOMER_ID CUSTOMER_NAME CITY PHONE EMAIL
-----
101 Ayesha HYDERABAD 9876543210 ayesha@gmail.com
102 Ruhi MUMBAI 9123456789 ruhi@gmail.com
103 Shaista HYDERABAD 9988776655 shaista@gmail.com
104 Arjun DELHI 9999999999 arjun@gmail.com
105 Saniya CHENNAI 9012345678 saniya@gmail.com
106 Meena PUNE 9871234567 meena@gmail.com
107 Saba BANGALORE 9765432109 saba@gmail.com
108 Divya KOLKATA 9999999999 divya@gmail.com
109 Pooja KOLKATA 9345678910 pooja@gmail.com
110 Aliya Hyderabad 9234567891 aliya@gmail.com
111 Farah Hyderabad 9876501234 farah@gmail.com

11 rows selected.

SQL> SELECT * FROM Accounts;

ACCOUNT_ID CUSTOMER_ID ACCOUNT_TYPE BALANCE
-----
201 101 Savings 60000
202 102 Current 70000
203 103 Current 25000
204 104 Savings 110000
205 105 Current 15000
206 106 Savings 82000
207 107 Current 43000
208 108 Savings 67000
209 109 Savings 120000
210 110 Current 35000

10 rows selected.

SQL> SELECT * FROM Customers WHERE City = 'Hyderabad';

CUSTOMER_ID CUSTOMER_NAME CITY PHONE EMAIL
-----
111 Farah Hyderabad 9876501234 farah@gmail.com

SQL>
```

```
Saniya 315 Deposit 6000 24-MAY-26
Meena 316 Withdraw 7000 25-MAY-26
Saba 317 Deposit 8500 26-MAY-26
Divya 318 Withdraw 2500 27-MAY-26
Pooja 319 Deposit 18000 28-MAY-26
Aliya 320 Withdraw 5500 29-MAY-26

20 rows selected.

SQL> SELECT C.customer_name,T.amount FROM Customers C INNER JOIN Accounts A ON C.customer_id = A.customer_id INNER JOIN Transactions T ON A.account_id = T.account_id WHERE T.amount > 10000;

CUSTOMER_NAME AMOUNT
-----
Meena 15000
Pooja 25000
Ayesha 12000
Pooja 18000

SQL> SELECT account_type, SUM(balance) AS total_balance FROM Accounts GROUP BY account_type;

ACCOUNT_TYPE TOTAL_BALANCE
-----
Current 188000
Savings 439000

SQL> SELECT SUM(balance) AS total_bank_balance FROM Accounts;

TOTAL_BANK_BALANCE
-----
627000

SQL> SELECT AVG(balance) AS average_balance FROM Accounts;

AVERAGE_BALANCE
-----
62700

SQL> SELECT MAX(amount) AS maximum_transaction FROM Transactions;

MAXIMUM_TRANSACTION
-----
25000
```

```
SQL> SELECT COUNT(*) AS total_customers FROM Customers;

TOTAL_CUSTOMERS
-----
11

SQL>
```

Module 7 – TCL Commands

Definition

TCL (Transaction Control Language) commands are used to manage transactions in database.

- **SAVEPOINT Example**

SAVEPOINT transfer_start;

- **ROLLBACK Example**

ROLLBACK TO transfer_start;

Explanation

ROLLBACK restores database to previous savepoint.

- **COMMIT Example**

COMMIT;

Explanation

COMMIT permanently saves changes.

EXECUTION



```
Run SQL Command Line
SQL*Plus: Release 10.2.0.1.0 - Production on Fri May 22 15:24:54 2020
Copyright (c) 1982, 2005, Oracle. All rights reserved.

SQL> conn
Enter user-name: system
Enter password:
Connected.
SQL> SAVEPOINT transfer_start;
Savepoint created.

SQL> UPDATE Accounts SET balance = balance - 5000 WHERE account_id = 101;
1 row updated.

SQL> UPDATE Accounts SET balance = balance + 5000 WHERE account_id = 102;
1 row updated.

SQL> SAVEPOINT amount_transferred;
Savepoint created.

SQL> rollback to amount_transferred;
Rollback complete.

SQL> commit;
Commit complete.

SQL>
```

Module 8 – DCL Commands

Definition

DCL (Data Control Language) commands are used to provide and remove user permissions.

- **GRANT Example**

```
GRANT SELECT, INSERT ON Customers TO finance_user;
```

Explanation

GRANT command gives permissions to users.

- **REVOKE Example**

```
REVOKE INSERT ON Customers FROM finance_user;
```

Explanation

REVOKE command removes permissions.

EXECUTION

```
SQL> CREATE USER finance_user IDENTIFIED BY finance123;
User created.

SQL> GRANT SELECT, INSERT ON Customers TO finance_user;
Grant succeeded.

SQL> REVOKE DELETE ON Customers FROM finance_user;
REVOKE DELETE ON Customers FROM finance_user
*
ERROR at line 1:
ORA-01927: cannot REVOKE privileges you did not grant

SQL> GRANT DELETE ON Customers TO finance_user;
Grant succeeded.

SQL> REVOKE DELETE ON Customers FROM finance_user;
Revoke succeeded.

SQL> GRANT SELECT ON ACCOUNTS TO finance_user;
Grant succeeded.

SQL>
```

Module 9 – ALTER & DROP Operations

Definition

ALTER command modifies existing tables. DROP command removes objects from database.

- **ALTER TABLE Example**

```
ALTER TABLE Customers ADD pan_number VARCHAR2(20);
```

- **DROP COLUMN Example**

```
ALTER TABLE Customers DROP COLUMN pan_number;
```

Explanation

DROP COLUMN removes column from table.

- **DROP USER Example**

```
DROP USER finance_user CASCADE;
```

Explanation

CASCADE removes user along with database objects.

EXECUTION

```
SQL> ALTER TABLE Customers ADD(pan_number VARCHAR2(20));
Table altered.
SQL> ALTER TABLE LOANS ADD(LOAN_STATUS VARCHAR2(20));
Table altered.
SQL> ALTER TABLE Customers rename column phone to mobile_number;
Table altered.
SQL> drop table employees;
Table dropped.
SQL>
```

Conclusion

Through this project, practical knowledge about Oracle SQL and Financial Management Systems was gained. Different concepts such as table creation, data insertion, joins, data cleaning, aggregate functions, transaction management, and permission management were understood.

This project improved understanding of real-world database applications used in banking and finance sectors. It also helped in improving SQL query writing, database design knowledge, and problem-solving skills. _

Improved practical knowledge of Oracle SQL and database management.

- Helped in understanding real-world financial and banking systems.
 - Enhanced SQL query writing and problem-solving skills.
 - Learned how to create and manage tables, records, and relationships.
 - Improved understanding of joins, aggregate functions, and transactions.
 - Developed knowledge about data security and permission management.
 - Helped in maintaining data accuracy and reducing data redundancy
- Increased confidence in handling large amounts of financial data

THANK YOU

Thank you for giving me the opportunity to complete this project.

This project helped in improving my SQL knowledge and understanding of database management concepts used in banking and financial systems

Submitted By

Shaista Anjum

Course

Data Science with GenAI